LAPORAN PRAKTIKUM

Blynk

Mata Kuliah ROBOTIKA

Nama Dino Rosalino Yuswanto

NPM 23081010152

Informatika Fakultas Ilmu Komputer

1. Ringkasan dan Target Pengujian

1.1 Ringkasan tugas

Praktikum dilakukan dengan mengintegrasikan arduino dengan Blynk.

1.2 Lingkungan pengujian

Komponen	Versi / Spesifikasi
Bahasa / framework	Arduino
Perangkat / simulator	ESP 32 & Blynk

1.3 Berkas yang diuji

Nama berkas	Peran / fungsi	Hubungan dengan berkas lain
Blynk.ino	Menggerakkan motor	Menerima data dari Blynk

2. Kode Program

2.1 Potongan kode utama

Kode blynk.ino

```
/*************************************************************
 * iMCLab + ESP32 + Blynk IoT
 * RPM Monitoring dan Kontrol PWM
 *************************************************************/

#define BLYNK_TEMPLATE_ID "TMPL6Zh3pGdg3"
#define BLYNK_TEMPLATE_NAME "Tes"
#define BLYNK_AUTH_TOKEN "MyyOkh_U32gbaUPcuatlwUkHuwvs7h7n"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>

//char ssid[] = "HOTSPOT@UPNJATIM.AC.ID";
//char pass[] = "belanegara";

//Konfigurasi Wifi
char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "AlookA";
char pass[] = "alookacafe012";


// ============================================================
// Motor dan LED
// ============================================================

int ledPin = 2;

int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

// ============================================================
// Sensor RPM
// ============================================================

const byte pin_rpm = 13;
const int HOLES = 2;

volatile unsigned int rev = 0;
volatile unsigned long lastPulseTime = 0;
```

```
const unsigned long DEBOUNCE_US = 500;

// ============================================================
// PWM
// ============================================================

const int freq = 30000;
const int pwmChannel = 0;
const int resolution = 8;

int dutyCycle = 200;

const int DUTY_STEP = 10;
const int DUTY_MIN = 0;
const int DUTY_MAX = 255;

bool motorEnable = false;

// ============================================================
// RPM
// ============================================================

float rpm = 0.0;
unsigned long old_time = 0;

// ============================================================

String inputString = "";

BlynkTimer timer;

// ============================================================
// ISR Encoder
// ============================================================

void IRAM_ATTR isr()
{
  unsigned long now = micros();

  if (now - lastPulseTime > DEBOUNCE_US)
  {
    rev++;
    lastPulseTime = now;
  }
}
```

```
// ============================================================
// Sync setelah reconnect Blynk
// ============================================================

BLYNK_CONNECTED()
{
  Blynk.syncVirtual(V2);
  Blynk.syncVirtual(V3);
}

// ============================================================
// Hitung RPM
// ============================================================

void hitungRPM()
{
  unsigned long cur_time = millis();

  if (cur_time - old_time >= 1000)
  {
    detachInterrupt(digitalPinToInterrupt(pin_rpm));

    float rot_per_sec = (float)rev / (float)HOLES;

    rpm = rot_per_sec * 60.0;

    rev = 0;

    attachInterrupt(
      digitalPinToInterrupt(pin_rpm),
      isr,
      FALLING);

    old_time = millis();
  }
}

// ============================================================
// Terapkan PWM ke Motor
// ============================================================

void applyMotor()
{
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  ledcWrite(pwmChannel, dutyCycle);
```

```
  digitalWrite(ledPin, dutyCycle > 0);

  // Sinkronisasi ke Blynk
  Blynk.virtualWrite(V1, dutyCycle);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

// ============================================================
// Status Serial
// ============================================================

void tampilkanStatus(const char* aksi)
{
  Serial.print(aksi);
  Serial.print(" | PWM=");
  Serial.print(dutyCycle);
  Serial.print(" | RPM=");
  Serial.println(rpm, 1);
}

// ============================================================
// Kirim Data ke Blynk
// ============================================================

void kirimKeBlynk()
{
  Blynk.virtualWrite(V0, rpm);
  Blynk.virtualWrite(V1, dutyCycle);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

// ============================================================
// Slider PWM dari Blynk (V2)
// ============================================================

BLYNK_WRITE(V2)
{
  dutyCycle = constrain(
    param.asInt(),
    DUTY_MIN,
    DUTY_MAX);

  motorEnable = (dutyCycle > 0);
```

```
  applyMotor();

  Serial.print("PWM dari Blynk = ");
  Serial.println(dutyCycle);
}

// ============================================================
// Tombol ON/OFF dari Blynk (V3)
// ============================================================

BLYNK_WRITE(V3)
{
  motorEnable = param.asInt();

  if (motorEnable)
  {
    if (dutyCycle == 0)
      dutyCycle = 200;
  }
  else
  {
    dutyCycle = 0;
  }

  applyMotor();

  Serial.print("Motor ");
  Serial.println(motorEnable ? "ON" : "OFF");
}

// ============================================================
// Setup
// ============================================================

void setup()
{
  Serial.begin(115200);

  pinMode(ledPin, OUTPUT);

  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(enable1Pin, OUTPUT);

  pinMode(pin_rpm, INPUT_PULLUP);

  attachInterrupt(
```

```
    digitalPinToInterrupt(pin_rpm),
    isr,
    FALLING);

  ledcSetup(
    pwmChannel,
    freq,
    resolution);

  ledcAttachPin(
    enable1Pin,
    pwmChannel);

  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  ledcWrite(pwmChannel, 0);

  old_time = millis();

//  Blynk.begin(
//    BLYNK_AUTH_TOKEN,
//    ssid,
//    pass);

  // Koneksi ke Blynk
  Blynk.begin(auth, ssid, pass);

  timer.setInterval(
    1000L,
    kirimKeBlynk);

  Serial.println("================================");
  Serial.println("iMCLab + Blynk Ready");
  Serial.println("================================");
  Serial.println("1 = ON");
  Serial.println("0 = OFF");
  Serial.println("+ = PWM naik");
  Serial.println("- = PWM turun");
  Serial.println("PWM:200");
  Serial.println("R = lihat RPM");
}

// ============================================================
// Loop
// ============================================================
```

```
void loop()
{
  Blynk.run();
  timer.run();

  hitungRPM();

  while (Serial.available() > 0)
  {
    char c = Serial.read();

    if (c == '\n' || c == '\r')
    {
      inputString.trim();

      if (inputString.startsWith("PWM:"))
      {
        int val = inputString.substring(4).toInt();

        val = constrain(
          val,
          DUTY_MIN,
          DUTY_MAX);

        dutyCycle = val;
        motorEnable = (dutyCycle > 0);

        applyMotor();

        tampilkanStatus("Set PWM");
      }

      else if (inputString.length() > 0)
      {
        char cmd = inputString.charAt(0);

        if (cmd == '1')
        {
          if (dutyCycle == 0)
            dutyCycle = 200;

          motorEnable = true;

          applyMotor();

          tampilkanStatus("Motor ON");
        }
```

```
    else if (cmd == '0')
    {
      dutyCycle = 0;
      motorEnable = false;

      applyMotor();

      Serial.println("Motor OFF");
    }

    else if (cmd == '+')
    {
      dutyCycle += DUTY_STEP;

      if (dutyCycle > DUTY_MAX)
        dutyCycle = DUTY_MAX;

      motorEnable = true;

      applyMotor();

      tampilkanStatus("PWM NAIK");
    }

    else if (cmd == '-')
    {
      dutyCycle -= DUTY_STEP;

      if (dutyCycle < DUTY_MIN)
        dutyCycle = DUTY_MIN;

      motorEnable = (dutyCycle > 0);

      applyMotor();

      tampilkanStatus("PWM TURUN");
    }

    else if (cmd == 'R' || cmd == 'r')
    {
      Serial.print("RPM = ");
      Serial.print(rpm, 1);
      Serial.print(" | PWM = ");
      Serial.println(dutyCycle);
    }
  }
```

```
      inputString = "";
    }
    else
    {
      inputString += c;
    }
  }
}
```

3. Penjelasan Kode

3.1 Penjelasan fungsi / blok

Fungsi / blok kode	Cara kerja	Input / output atau efek
Konfigurasi Wi-Fi dan Blynk	Menetapkan template Blynk, token autentikasi, SSID, dan kata sandi Wi-Fi. Library Wi-Fi dan Blynk digunakan untuk menghubungkan ESP32 ke layanan Blynk.	Input: kredensial koneksi. Output: koneksi ke Blynk.
Konfigurasi pin motor dan LED	GPIO 27 dan GPIO 26 mengatur arah motor; GPIO 12 memberikan sinyal PWM ke driver; GPIO 2 digunakan sebagai indikator LED.	Output: sinyal arah motor, PWM, dan LED.
Sensor RPM dan ISR	GPIO 13 menerima pulsa sensor. Fungsi isr() menghitung pulsa pada tepi FALLING dan menerapkan jeda debounce 500 mikrodetik.	Input: pulsa sensor. Output: penghitung pulsa rev.
Penghitungan RPM	Setiap sekitar satu detik, jumlah pulsa dibagi HOLES (2), lalu dikalikan 60 untuk memperoleh RPM.	Output: nilai RPM pada variabel rpm.
PWM dan applyMotor()	PWM dikonfigurasi pada frekuensi 30 kHz dengan resolusi 8-bit. applyMotor() menerapkan duty cycle dan memperbarui LED serta nilai virtual Blynk.	Input: dutyCycle 0–255. Output: PWM dan status dashboard.
Kontrol Blynk	BLYNK_WRITE(V2) membaca nilai slider PWM. BLYNK_WRITE(V3) membaca tombol ON/OFF dan mengubah duty cycle.	V2: nilai PWM. V3: perintah ON/OFF.
Pengiriman data Blynk	kirimKeBlynk() dijalankan oleh timer setiap 1.000 ms untuk mengirim RPM, PWM, dan status motor.	V0 = RPM; V1/V2 = PWM; V3 = status berdasarkan PWM.
Kontrol Serial	Karakter disimpan hingga newline atau carriage return. Program mengenali PWM:angka, 1, 0, +, -, serta R/r.	Input: perintah Serial. Output: perubahan PWM atau status RPM.
Konfigurasi Wi-Fi dan Blynk	Menetapkan template Blynk, token autentikasi, SSID, dan kata sandi Wi-Fi. Library Wi-Fi dan Blynk digunakan untuk menghubungkan ESP32 ke layanan Blynk.	Input: kredensial koneksi. Output: koneksi ke Blynk.

3.2 Alur eksekusi

Alur Inisialisasi :

1. ESP32 memulai komunikasi Serial pada baud rate 115200 dan mengatur pin LED, motor, serta sensor RPM.

2. Interrupt dipasang pada GPIO 13 dengan pemicu FALLING. PWM dikonfigurasi pada frekuensi 30.000 Hz dan resolusi 8-bit.
3. Keluaran PWM diatur ke 0 sehingga motor dimulai dalam keadaan mati.
4. ESP32 mencoba terhubung ke Wi-Fi dan Blynk menggunakan kredensial yang ditetapkan dalam kode.
5. Timer diatur untuk mengirim data ke Blynk setiap satu detik.

Alur saat program berjalan :

1. ESP32 memulai komunikasi Serial pada baud rate 115200 dan mengatur pin LED, motor, serta sensor RPM.
2. Interrupt dipasang pada GPIO 13 dengan pemicu FALLING. PWM dikonfigurasi pada frekuensi 30.000 Hz dan resolusi 8-bit.
3. Keluaran PWM diatur ke 0 sehingga motor dimulai dalam keadaan mati.
4. ESP32 mencoba terhubung ke Wi-Fi dan Blynk menggunakan kredensial yang ditetapkan dalam kode.
5. Timer diatur untuk mengirim data ke Blynk setiap satu detik.

4. Hasil Eksekusi dan Pengujian

FOTO HASIL 1

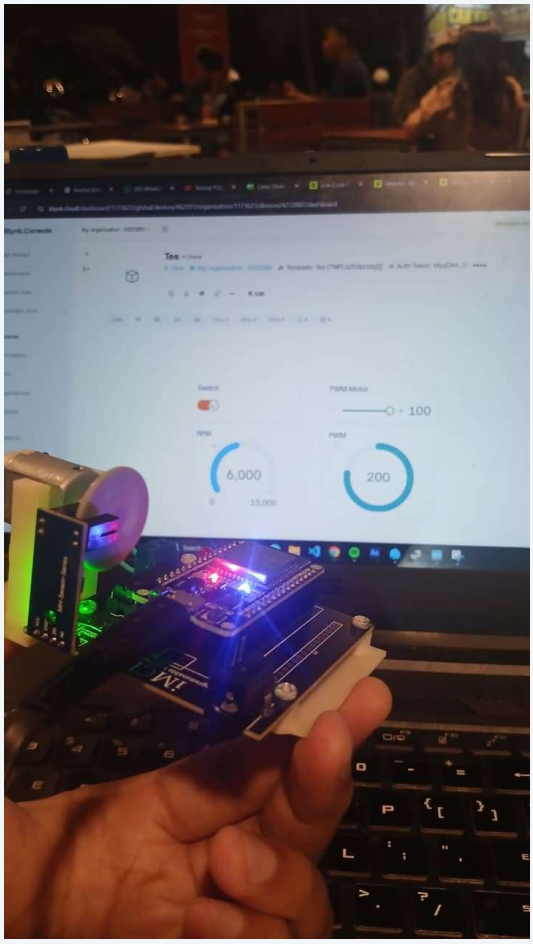
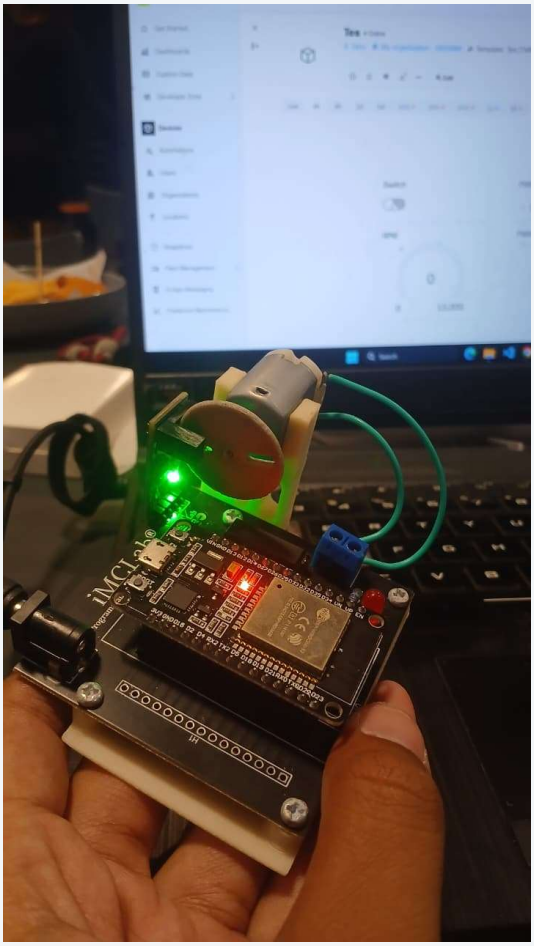


FOTO HASIL 2



5. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, motor DC dikendalikan menggunakan ESP32 yang terintegrasi dengan platform Blynk IoT melalui koneksi Wi-Fi. Pengendalian motor dilakukan dengan memanfaatkan tombol ON/OFF dan slider untuk mengatur nilai duty cycle Pulse Width Modulation (PWM), sehingga tingkat daya yang diberikan kepada motor dapat disesuaikan. Selain itu, praktikum ini menerapkan sensor RPM untuk menghitung kecepatan putaran motor berdasarkan pulsa yang diterima oleh ESP32. Data RPM, nilai PWM, dan status motor kemudian dikirimkan ke dashboard Blynk agar dapat dipantau secara berkala. Pengendalian juga dapat dilakukan melalui komunikasi serial dengan perintah untuk menyalakan dan mematikan motor, menaikkan atau menurunkan nilai PWM, serta menampilkan informasi RPM. Dengan demikian, praktikum ini memberikan pemahaman mengenai integrasi ESP32, kontrol motor DC berbasis PWM, pemantauan RPM, dan penerapan Internet of Things (IoT) menggunakan Blynk sebagai antarmuka pemantauan dan pengendalian motor.